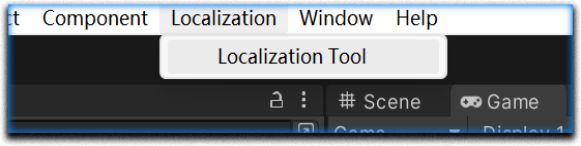
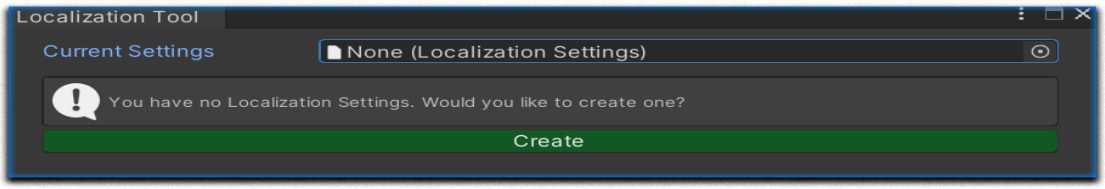
USER GUIDE

GETTING STARTED

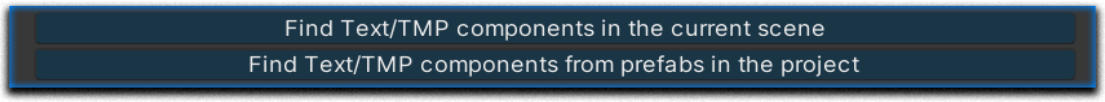
1. Open the **Localization Tool**



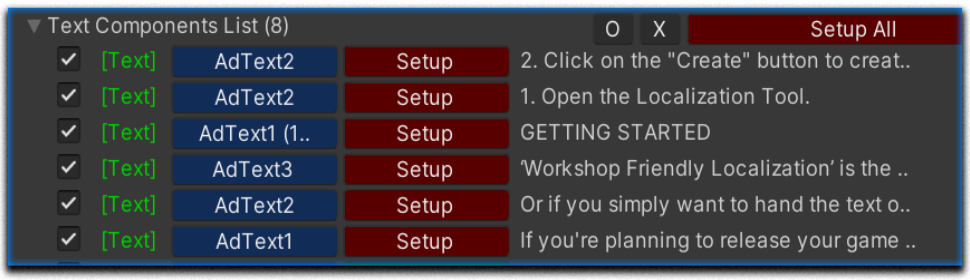
2. Navigate to the current settings file and delete it, then go back to **Localization Tool** and click on the **Create** button to create your own Localization Setting Asset.



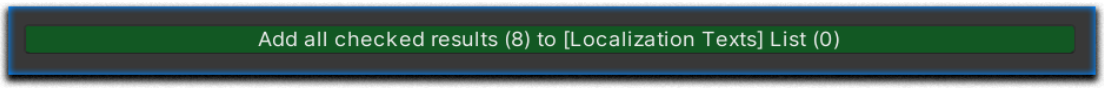
3. Click these two buttons: the first will find all Text/TMP components in the current scene, and the second will find them in prefabs across the entire project. You'll need to open each scene individually and use the first button to locate all Text/TMP components in those scenes.



4. Expand the **Text Components List** to view all the text components that were found. Click the **Setup All** button to configure them for localization.



5. Click this button to add the text components to your settings file for localization.

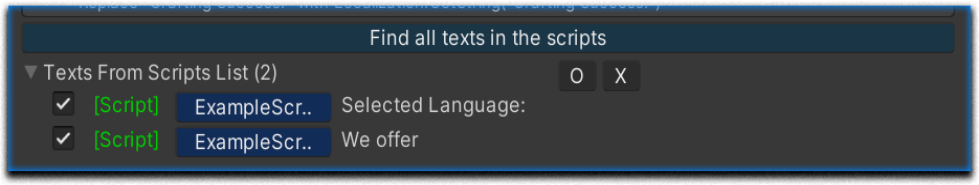


6. Open your scripts and locate all the hardcoded strings that require localization. Replace each string with Localization.GetString(Your string) as shown in the example below:

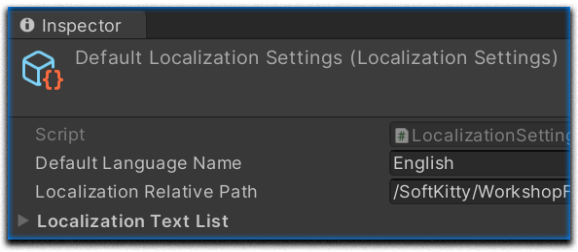
```
LanguageName.text = "Selected Language:"+ Languages[Localization.SelectedLanguage];
LanguageCount.text = "We offer "+ Languages.Length.ToString() + " selectable languages in the demo.";
```

```
LanguageName.text = Localization.GetString("Selected Language:")+ Languages[Localization.SelectedLanguage];
LanguageCount.text = Localization.GetString("We offer ")+ Languages.Length.ToString() + " selectable languages in the demo.";
```

7. Return to the **Localization Tool** and click on the **Find all texts in the scripts** button to locate the strings you just marked. After that, click on the **Add all checked results to [Localization Texts] List** button to include them in your localization settings file.

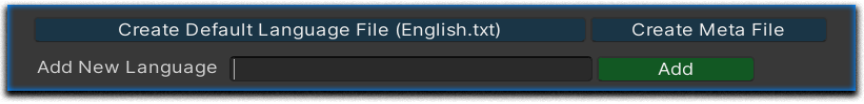


8. Expand the **Localization Files** category, then set up your default language by clicking the button to navigate to the settings file. Once there, change the default language setting in the Inspector.



9. Set up the relative path for the localization files. This path is relative to your Application.dataPath. If you want a different path for the build, you can configure it in your code by calling Localization.ReadLocalizationFiles(LocalizationSettings.GetLocalizationPath())

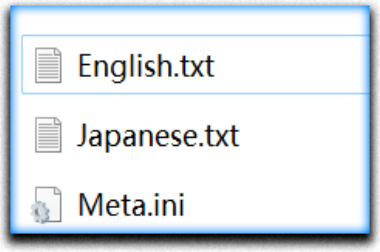
10. Click the **Create Default Language File** button and the **Create Meta File** button to generate the base localization files. After that, you can add additional languages using the **Add New Language** field.



11. Whenever you added new texts to localization list, a green **Update** button will show behind the localization files, click them to update the files with the new texts you added.

12. Add this line to the Awake method in a script within your first scene: string [] languages = Localization.ReadLocalizationFiles(localizationPath); This will return the language names as a string array, which you can use to populate language selection options in your game.

13. Now that everything is set up, you can send those localization text files to AI for translation. Additionally, players can contribute to localization by modifying the existing text files or creating their own and placing them into the same folder. The system will automatically read all the text files in that folder and use the file names as the Language Name options.



TIPS

Check \Assets\SoftKitty\WorkshopFriendlyLocalization\ExampleScene\Scripts\ExampleScript.cs to learn how to make Languages DropDown options and how to integrate the localization with your script.

Unity Legacy <Text> natively supports different Languages, but <TextMeshPro> does not. If you have to use TMP, then you have to manually setup your font file to support different Languages.